

EN.601.447/647 Computational Genomics: Sequences

Homework 2 (Fall 2026)

Out of 41 points (21 written, 20 programming).

Programming questions

1. Quality control. (4 pts)

Submit this question as `hw2q1`, with the appropriate suffix: `.py`, `.java` or `.cpp`.

Download these three FASTQ datasets, which we will call A, B and C:

https://www.cs.jhu.edu/~langmea/teaching/f2026/hw2/hw2q1_A.fastq

https://www.cs.jhu.edu/~langmea/teaching/f2026/hw2/hw2q1_B.fastq

https://www.cs.jhu.edu/~langmea/teaching/f2026/hw2/hw2q1_C.fastq

Each contains 10,000 DNA sequencing reads from a real whole-genome sequencing project. The species being sequenced is different in each case. We modified the data so that all three share the same read length and base quality scale.

Recall that a single *read* comes from one *cluster* of single-stranded DNA clones on the slide. One 4-line FASTQ record corresponds to one read/cluster. During one *sequencing cycle*, the sequencer reads the next base from all the clusters. Thus, one sequencing cycle corresponds to one *position* across all the reads. This notebook has more detail and some useful code:

<https://t.jh.edu/rfHz1y>.

Write a program that reads a FASTQ file and summarizes information about the file. Your program will take two command-line arguments. The first will be the name of the FASTQ file, the second will be the name of the output summary file. So if you are using Python and testing your program with a FASTQ file called `reads.fastq` and the output file called `summary.txt`, you could run your program like this:

```
python3 hw2q1.py reads.fastq summary.txt
```

The summary has one number per sequencing cycle, one per line, in cycle order. The number for a cycle is how many bases at that position, counted over every read in the file, have a quality value (denoted Q in lecture) less than 10. Assume all the reads in a file are the same length; so if the reads are 60 bases long your program will output 60 lines. The first line of output is the first cycle, meaning the leftmost position in every read; the second line is the second cycle / second-from-the-left position, and so on.

Note that written question 1(c) asks you about this program's output when run on the three FASTQ files linked above.

For the following FASTQ file:

```

@r1
TCGATA
+
$HHH#H
@r2
CCCCNG
+
IIII!I
@r3
GATTCA
+
BDDH(H
@r4
ACGTAC
+
HHHHH*
@r5
TTGGCA
+
AAAAHH

```

the reads are 6 bases long, so the summary has six lines and should read:

```

1
0
0
0
3
1

```

The fifth cycle is the worst one in this example. Three of the five reads have $Q < 10$ in that cycle. The first (leftmost) cycle has only one low-quality base and the last (rightmost) cycle also has one. The others have none. This example is given as a “visible” Gradescope test.

Programming Q2–Q4

The next three programming problems build on one another.

Q2: Your program will build a k -mer index of a genome from a FASTA file.

Q3: Your program will use a k -mer index to find, for each read in a FASTQ file, every place the read occurs with at most three mismatches.

Q4: Your program will use a k -mer index to decide, for each read, whether it occurs anywhere allowing one mismatch *or* one gap.

2. Building a k -mer index. (2 pts)

Submit this question as `hw2q2`, with the appropriate suffix: `.py`, `.java` or `.cpp`.

This question focuses on your index-building code, which you can reuse in later questions. Download this file containing the genome of $\Phi X174$, a virus first sequenced by Fred Sanger and colleagues in 1977:

<https://www.cs.jhu.edu/~langmea/resources/phix.fa>

In class, we discussed k -mer indexes. Given a text string, we take all its substrings of length k and insert them into a data structure (e.g. hash table) that associates the k -mer with the offsets where it occurs.

Write a program that builds a k -mer index of a genome in a FASTA file for a given value of k . It will take 3 arguments from the command line:

```
python3 hw2q2.py test.fa 3 output.txt
```

where `test.fa` is the FASTA file containing the genome, the second argument (3 in this case) is the value of k to use, and `output.txt` is the file that gets the output. Your program will write two values separated by a space to the output file: the total number of distinct keys in the k -mer index and the number of keys that appear exactly once.

You may assume the input is well formed. The FASTA will have a single record, the sequence will contain only uppercase A, C, G and T letters (possibly wrapped across several lines), and k is a positive integer no larger than the length of the sequence. All of this is true for the $\Phi X174$ FASTA example:

INPUT in test.fa:

```
>ref
GATTA
CAGAT
```

and k given on the command line as 3:

*The index this builds is shown below to explain where the two numbers come from. It is **not** what your program writes:*

```
GAT: [0, 7]
ATT: [1]
TTA: [2]
TAC: [3]
ACA: [4]
CAG: [5]
AGA: [6]
```

Seven distinct 3-mers, six of which occur exactly once (GAT appears twice). So the two numbers are 7 and 6, and the whole of the output file is:

Expected OUTPUT in output.txt:

3. Approximate matching with up to three mismatches. (7 pts)

Submit this question as `hw2q3`, with the appropriate suffix: `.py`, `.java` or `.cpp`.

A sequencing read can fail to have an exact match to the reference genome even when it “truly” originated from an organism with a similar genome. Therefore, we need approximate matching. In lecture we covered the *pigeonhole principle*:

Let $p_1, p_2, \dots, p_{x+1}$ be a partitioning of P into $x + 1$ non-empty, non-overlapping substrings. If P occurs in T with at most x mismatches, then at least one of the partitions matches exactly.

Write a program that reports, for each read in a FASTQ file, every offset in the genome where that read occurs with at most 3 mismatches.

Every read is 20 nt and $x = 3$, so the principle asks for $x + 1 = 4$ partitions, and $20/4 = 5$, making each partition a 5-mer. Build a 5-mer index of the genome, perform an index lookup for each of the read’s four partitions, and check in the neighborhood of the index hits for full approximate matches. Report each match offset once only.

Your program takes the usual three command-line arguments: the FASTA file, the FASTQ file, and the output file.

Output. One line per read, in the order the reads appear in the FASTQ file. Each line holds the offsets where that read occurs with at most 3 mismatches, in ascending order, separated by single spaces. A read with no occurrence gets an empty line. Offsets are 0-based from the start of the genome sequence.

Example. If the FASTA holds

```
>reference sequence
TAGGGCTGGAGGCATTACGAATTAATAATTGGGTCGAGCATTAATC
```

and the FASTQ file holds three reads:

```
@r1
CATTACGAATTAATAATTGG
+
IIIIIIIIIIIIIIIIIIII
@r2
CCGTACGAATTGATAATTTG
+
IIIIIIIIIIIIIIIIIIII
@r3
CACTGCGAATTAATAATTAG
+
IIIIIIIIIIIIIIIIIIII
```

then the output is

```
12
```

```
12
```

The first read occurs exactly at offset 12. The second differs from the genome at that offset in four positions and occurs nowhere else, so its line is empty. The third read differs at that same offset in three positions; that's close enough, so it is reported.

You may assume the input is well behaved: the FASTA holds a single record whose sequence uses only uppercase A, C, G and T though it may be wrapped across lines, and every read in the FASTQ is 20 nt.

Match the reads as they are given, against the genome as it is given. Ignore the reverse complement; you do not have to consider the other strand.

4. **Approximate matching with one mismatch or one gap.** (7 pts)

Submit this question as hw2q4, with the appropriate suffix: .py, .java or .cpp.

The previous question allowed mismatches only. In reality, differences between a read and a reference could also include *gaps*, i.e. insertions and deletions. For this question we look for approximate matches with up to one difference, where a difference is either a *mismatch* (one base replaced by another) or a single-base *gap* (a base present in the genome but not in the read, or the other way round). Another way to say this is that we are searching for substrings of T such that P matches the substring within an edit distance of 1.

The budget is one difference in total. Two mismatches, or a mismatch and a gap together, is over budget, and a run of two missing bases counts as two differences rather than one.

Suppose a stretch of the genome reads TACGTA. Lining three reads up against the ACGT inside it:

genome	A C G T	genome	A C G T	genome	A C G T
read	A C T T	read	A - G T	read	A - G A
one mismatch, so ACTT occurs		one gap, the C unmatched, so AGT occurs		a gap and a mismatch, so AGA does not occur here	

Notice what a gap does to lengths: the read AGT is 3 bases long but it covered 4 bases of the genome. That is why, for a 20 nt read, the stretch of genome you compare against may be 19, 20 or 21 bases rather than always 20.

The pigeonhole principle still applies. Since one difference is allowed, it asks for two partitions, and since every read is 20 nt, each partition is a 10-mer. Build a 10-mer index of the genome and use it to find the places worth checking.

Gaps are tricky. If the read matches a stretch of genome one character longer or shorter than itself, then the offset implied by a partition hit can be off by one, and the stretch you compare against is 19, 20 or 21 characters rather than always 20. Your candidate checking has to allow for both.

Your program takes the same three command-line arguments as the previous question, and the same assumptions hold. As before, match the reads as they are given and ignore the reverse complement.

Output. One line per read, in the order the reads appear in the FASTQ file. Write 1 if the read occurs anywhere in the genome under the definition above and 0 if it does not. You do not have to report where.

Example. With the same genome as the previous question, and a FASTQ file in the same four-line-per-record format holding these four read sequences in this order,

```
CATTACGAATTAATAATTGG
CATTACGCATTAATAATTGG
CATGACGAATTAATACTTGG
CACTGGAGCAGCCCTTAGTA
```

the output is

```
1
1
0
0
```

The first read occurs in the text exactly, so it trivially has a match within an edit distance of 1. The second read differs from the genome in one position, which is a substitution. The third differs in two positions; it cannot be made to match the text with 1 edit (it needs 2), so it is outside the budget and we report 0. The fourth does not come from this genome at all. Note that this example doesn't include a read that matches with exactly 1 gap, so you should probably make such a test yourself.

Written questions

1. Base calling and FASTQ. (6 pts)

In sequencing by synthesis, a cluster of cloned template molecules is photographed once per cycle. The base call depends on which color dominated the light coming from the cluster. The base *quality* conveys the sequencer's confidence level in its call. Here is one record from a FASTQ file, with the four lines labeled:

```
1 @ERR194146.1 HSQ1008:141:D0CC8ACXX:3:1308:20201:36071/1
2 ACATCTGGTTCC
3 +
4 IIIIIIIIIII5+
```

- (a) (2 pts) At one cycle a cluster emits light measured as: A 0 units, C 6 units, G 3 units, T 0 units. Give the base call, the estimated probability p that the call is wrong, the quality $Q = -10 \log_{10} p$, and the character this would be written as in a FASTQ file using the Phred+33 encoding. Show your working. Estimate p as the fraction of the cluster's light that did *not* come from the base you called, and round Q to the nearest integer before turning it into an ASCII character.
- (b) (2 pts) The quality values for the read above are Phred+33 encoded. We discussed this encoding in lecture, and it is also discussed here: <https://t.jh.edu/rfHz1y>. Give Q and the estimated error probability p for the *first* base and also for the *last* base. Also, answer: Based on how the technology works, why is it expected that later bases will generally have lower quality?
- (c) (2 pts) Run your `hw2q1` program from programming question 1 on each of the three datasets A, B and C. In one of them, a single sequencing cycle went badly wrong: a majority of the reads have a low-quality base there, far more than at any other cycle and far more than at the cycles just before or after it. Say which dataset has this aberrant cycle, which cycle it is, and how many low-quality bases your program counted there. Name the cycle by counting the leftmost position as the first cycle; recall that the first line of output from our program is the first cycle. It's not necessary, but you might consider plotting the three outputs to find this outlier.

2. Canonical k -mers. (4 pts)

The DNA molecule is double stranded. We did not worry about double-strandedness in the programming questions, so let's think about it here.

For every k -mer on one strand, there is a reverse-complemented counterpart k -mer on the other. Some algorithms want to compare two sequences based on their k -mers, but without really caring which strand the sequences came from; i.e. the algorithm wants to treat a k -mer and its reverse complement as essentially the same.

Many methods handle this by first *canonicalizing* the k -mers. A k -mer is canonical if it is lexicographically less than or equal to its reverse complement. To canonicalize the k -mers of a sequence, we do the following. For each k -mer, we leave it alone if it is already canonical. Otherwise, we replace the k -mer with its reverse complement.

Say we want to canonicalize all the 3-mers of the string ACGTA. First we consider its 3-mers: ACG, CGT, and GTA.

- The first 3-mer ACG is less than its reverse complement CGT, so we leave it alone.
- The second 3-mer CGT is greater than its reverse complement ACG, so we replace it with its reverse complement, ACG.
- The third 3-mer GTA is less than its reverse complement TAC, so we leave it alone.

And get the canonical k -mers: ACG, ACG, GTA.

- (a) (2 pts) Given the following FASTA sequence, how many *distinct* canonical k -mers are there for $k = 4$?

```
>example
ATGCATGCATGC
```

- (b) (2 pts) Given the same FASTA sequence, how many *distinct* canonical k -mers are there for $k = 5$?

3. **Offline and approximate matching.** (3 pts)

- (a) (2 pts) A sequencing read is a short string drawn from someone's genome. We usually refer to the individual being sequenced as the "donor." The reference genome we use is derived from some other individual, not the donor. Give two *distinct* reasons a read sequenced from the donor by a real DNA sequencer might fail to match the reference exactly.
- (b) (1 pt) Say whether the approach you implemented in Questions 3 and 4 of the programming section is *online* or *offline*, and describe the strategy you used in both of them in one or two sentences.

4. **Boyer-Moore skip amounts.** (4 pts)

Boyer-Moore compares P with the text right to left and, on a mismatch, asks two rules how far P may shift. Apply the bad character rule and the good suffix rule exactly as they are defined on the course slides.

Let's clarify two terms that can be confusing: *shifts* and *skips*. The naive exact matching algorithm always *shifts* by 1 in its outer loop. In other words, it never *skips* any alignments or, equivalently, always *skips* 0 alignments.

When a Boyer-Moore skipping rule tells you to "skip 1 alignment," it's telling you to shift by $1+1 = 2$ alignments. Skipping 2 alignments means shifting by $2+1 = 3$ alignments, etc. In short, the number of shifts is always 1 plus the number of skips. Please keep this in mind.

Throughout this question $P = \text{CATCTC}$. The two alignments below are separate scenarios, i.e. separate iterations of the outer loop. In each case, we are considering only P and the six-letter substring of T that P is sitting "below." Offsets are 0-based.

- (a) (2 pts) P sits below the substring **GCCGTC** from T :

T	...	G	C	C	G	T	C	...
P		C	A	T	C	T	C	
offset		0	1	2	3	4	5	

Scanning right to left, give the offset j of the first mismatch and the text character c there. Then give the number of alignments skipped by the bad character rule, and the number of alignments skipped by the good suffix rule, with a sentence of reasoning for each.

- (b) (2 pts) P sits below the substring **TTACTC** from T :

T	...	T	T	A	C	T	C	...
P		C	A	T	C	T	C	
offset		0	1	2	3	4	5	

Answer the same question as the previous part.

5. Natural or engineered. (4 pts)

In August 2026, King and colleagues reported in *Science* the first viable phage (a virus that infects bacteria) designed by a genome language model. They fine-tuned the models on *Microviridae* sequences, repeatedly prompted the models with a small part of the $\Phi X174$ genome, and ultimately synthesized roughly three hundred new genomes. They found that sixteen of these produced infectious phage in *E. coli*, i.e. they basically “worked as they should”.

But if a sequence can be designed by an AI language model, then an increasingly important problem is to be able to tell the difference between the natural and AI-designed versions of a virus. For biodefense the question “did this come from nature” is crucial. This question is a small version of that problem. First download these files:

- the natural $\Phi X174$ genome, the control that has been run on sequencers for decades:
<https://www.cs.jhu.edu/~langmea/resources/phix.fa>
- Evo- $\Phi 63$, one of the sixteen viable generated genomes from that paper:
https://www.cs.jhu.edu/~langmea/teaching/f2026/hw2/evo_phage.fa
- a set of 200 simulated 20 nt reads:
https://www.cs.jhu.edu/~langmea/teaching/f2026/hw2/mystery_reads.fastq

All of the reads in `mystery_reads.fastq` came from one of the two genomes, but we’re not telling you which. Either they all came from `phix.fa` or they all came from `evo_phage.fa`. The reads are not perfect substrings of the genome they came from, because we added synthetic sequencing errors at a rate of about 2%. As in the programming questions, match the reads as they are given and ignore the reverse complement.

Do an analysis to determine which genome the reads came from. You can use your pattern matching code from the programming section, or another tool if you like. Explain which tools you used, which measurements you made, and why you reached the conclusion you did.

6. Special review problem: skip-rule puzzles. (0 pts)

This problem is not graded. A problem like this one will appear on Midterm 1. Solutions will be posted; try it before you read them.

Let $P = \text{TGCTGC}$, and let W be the six-letter substring of T that P is sitting “below.” Consider the usual Boyer-Moore strategies we discussed in class. We compare characters right to left and apply the bad character rule and the good suffix rule just as we defined them in lecture. One convention is worth stating outright, because this problem depends on it: P never moves by less than one position, so when the very first comparison mismatches and no part of P has matched yet, the good suffix rule has nothing to say and Boyer-Moore shifts by 1 unless the bad character rule asks for more.

- (a) Give a sequence for W for which Boyer-Moore shifts P by only 1, i.e. it skips no alignments at all.
- (b) Give a W for which both rules propose *the same* shift of 3 (skipping 2 alignments). Briefly explain why each rule skips 2 alignments.
- (c) For part (b), how many different W 's are there such that both rules skip 2 alignments? Answer without enumerating them all.